In an operating system (OS), a thread is the smallest unit of execution that can be managed independently. Threads are used to allow processes to perform multiple tasks simultaneously.

A thread refers to a single sequential flow of activities being executed in a process; it is also known as the thread of execution or the thread of control. Now, thread execution is possible within any OS's process. Apart from that, a process can have several threads.

Thread in Operating System

[redacted]

A thread is a single sequence stream within a process. Threads are also called **lightweight processes** as they possess some of the properties of processes. Each thread belongs to exactly one process.

- In an operating system that supports multithreading, the process can consist of many threads. But threads can be effective only if the CPU is more than 1 otherwise two threads have to context switch for that single CPU.
- All threads belonging to the same process share – code section, data section, and OS resources (e.g. open files and signals)
- But each thread has its own (thread control block) – thread ID, program counter, register set, and a stack
- Any operating system process can execute a thread. we can say that single process can have multiple threads

Why Do We Need Thread?

- Threads run in concurrent manner that improves the application performance. Each such thread has its own CPU state and stack, but they share the address space of the process and the environment. For example, when we work on Microsoft Word or Google Docs, we notice that while we are typing, multiple things happen together (formatting is applied, page is changed and auto save happens).
- Threads can share common data so they do not need to use inter-process communication. Like the processes, threads also have states like ready, executing, blocked, etc.
- Priority can be assigned to the threads just like the process, and the highest priority thread is scheduled first.

- Each thread has its own Thread Control Block (TCB). Like the process, a context switch occurs for the thread, and register contents are saved in (TCB). As threads share the same address space and resources, synchronization is also required for the various activities of the thread.

Introduction of Process Synchronization

[redacted]

Process Synchronization is used in a computer system to ensure that multiple processes or threads can run concurrently without interfering with each other.

The main objective of process synchronization is to ensure that multiple processes access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access. To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.

In a multi-process system, synchronization is necessary to ensure data consistency and integrity, and to avoid the risk of deadlocks and other synchronization problems. Process synchronization is an important aspect of modern operating systems, and it plays a crucial role in ensuring the correct and efficient functioning of multi-process systems.

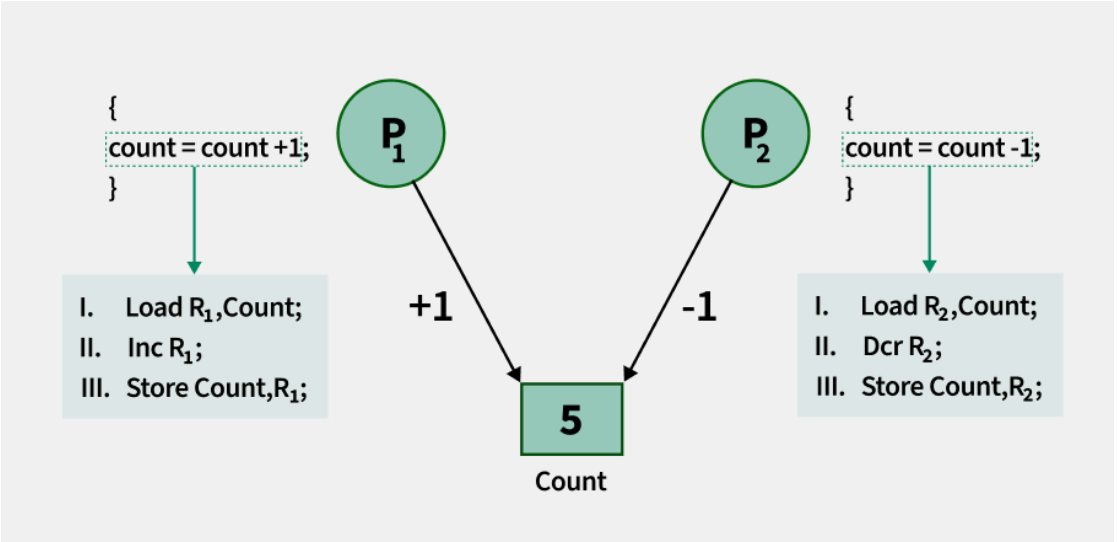
On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process**: The execution of one process does not affect the execution of other processes.
- **Cooperative Process**: A process that can affect or be affected by other processes executing in the system.

Process synchronization problem arises in the case of Cooperative processes also because resources are shared in Cooperative processes.

Process Synchronization

Process Synchronization is the coordination of execution of multiple processes in a multi-process system to ensure that they access shared resources in a controlled and predictable manner. It aims to resolve the problem of race conditions and other synchronization issues in a concurrent system.



Lack of Synchronization in Inter Process Communication Environment leads to following problems:

1. **Inconsistency:** When two or more processes access shared data at the same time without proper synchronization. This can lead to conflicting changes, where one process's update is overwritten by another, causing the data to become unreliable and incorrect.
2. **Loss of Data:** Loss of data occurs when multiple processes try to write or modify the same shared resource without coordination. If one process overwrites the data before another process finishes, important information can be lost, leading to incomplete or corrupted data.
3. **Deadlock:** Lack of Synchronization leads to Deadlock which means that two or more processes get stuck, each waiting for the other to release a resource. Because none of the processes can continue, the system becomes unresponsive and none of the processes can complete their tasks.

Types of Process Synchronization

The two primary type of process Synchronization in an Operating System are:

1. **Competitive:** Two or more processes are said to be in Competitive Synchronization if and only if they compete for the accessibility of a shared resource.
   Lack of Synchronization among Competing process may lead to either Inconsistency or Data loss.
2. **Cooperative:** Two or more processes are said to be in Cooperative Synchronization if and only if they get affected by each other i.e. execution of one process affects the other process.
   Lack of Synchronization among Cooperating process may lead to Deadlock.

Example:
Let consider a Linux code:

```
>>ps/grep "chrome"/wc
```

- ps command produces list of processes running in linux.
- grep command find/count the lines form the output of the ps command.
- wc command counts how many words are in the output.

Therefore, three processes are created which are ps, grep and wc. grep takes input from ps and wc takes input from grep.
From this example, we can understand the concept of cooperative processes, where some processes produce and others consume, and thus work together. This type of problem must be handled by the operating system, as it is the manager.

Conditions That Require Process Synchronization

1. **Critical Section:** It is that part of the program where shared resources are accessed. Only one process can execute the critical section at a given point of time. If there are no shared resources, then no need of synchronization mechanisms.
2. **Race Condition:** It is a situation wherein processes are trying to access the critical section and the final result depends on the order in which they finish their update. Process Synchronization mechanism need to ensure that instructions are being executed in a required order only.
3. **Pre Emption:** Pre-emption is when the operating system stops a running process to give the CPU to another process. This allows the system to make sure that important tasks get enough CPU time. This is important as mainly issues arise when a process has not finished its job on shared resource and got preempted. The other process might end up reading an inconsistent value if process synchronization is not done.

What is Race Condition?

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple process/thread execution in the critical section differs according to the order in which the threads execute. Race conditions in critical sections can be avoided if the critical section is treated as an atomic instruction. Also, proper thread synchronization using locks or atomic variables can prevent race conditions.
Let us consider the following example.

- There is a shared variable balance with value 100.
- There are two processes deposit (10) and withdraw (10). The deposit process does balance = balance + 10 and withdraw process does balance = balance – 10.
- Suppose these processes run in an interleaved manner. The deposit() fetches the balance as 100, then gets pre-empted.
- Now withdraw () get scheduled and makes balance 90.

- Finally, deposit is rescheduled and makes the value as 110. This value is not correct as the balance after both operations should be 100 only

We cannot notice that the different segments of two processes running in different order would give different values of balance.

Critical Section Problem

A *critical section* is a code segment that can be accessed by only one process at a time. The critical section contains shared variables that need to be synchronized to maintain the consistency of data variables. So the critical section problem means designing a way for cooperative processes to access shared resources without creating data inconsistencies.

In the above example, the operations that involve balance variable should be put in critical sections of both deposit and withdraw.

```
do {
    entry section
        critical section
    exit section
        remainder section
  } while (TRUE);
```

In the entry section, the process requests for entry in the **Critical Section.**

Any solution to the critical section problem must satisfy three requirements:

- **Mutual Exclusion**: If a process is executing in its critical section, then no other process is allowed to execute in the critical section.
- **Progress**: If no process is executing in the critical section and other processes are waiting outside the critical section, then only those processes that are not executing in their remainder section can participate in deciding which will enter the critical section next, and the selection cannot be postponed indefinitely.
- **Bounded Waiting**: A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Critical section Problem –

Classical IPC Problems

Various classical Inter-Process Communication (IPC) problems include:

- Producer Consumer Problem
- Readers-Writers Problem
- Dining Philosophers Problem

Producer Consumer Problem

The Producer-Consumer Problem is a classic example of process synchronization. **It describes a situation where two types of processes producers and consumers share a common, limited-size buffer or storage.**

- **Producer**: A producer creates or generates data and puts it into the shared buffer. It continues to produce data as long as there is space in the buffer.

- **Consumer**: A consumer takes data from the buffer and uses it. It continues to consume data as long as there is data available in the buffer.

The challenge arises because both the producer and the consumer need to access the same buffer at the same time, and if they do not properly synchronize their actions, issues can occur.
If the shared variable is not placed in critical section , then may create problem wrt synchronisation

Key Problems in the Producer-Consumer Problem:

1. **Buffer Overflow**: **If the**$Type\ equation\ here.$ **producer tries to add data when the buffer is full**, there will be no space for new data, causing the producer to be blocked.

2. **Buffer Underflow: If the consumer tries to consume data when the buffer is empty,** it has nothing to consume, causing the consumer to be blocked.

Producer-Consumer Problem – Solution (using Semaphores)

In an **Operating System (OS)**, **Supervisor Mode** is a special **privileged mode of the CPU** in which the operating system has full control over the computer system.

Supervisor Mode (also called **Kernel Mode** or **Privileged Mode**) is the mode in which the **Operating System executes its core functions** and can access **all hardware resources and system memory directly**.

In this mode, the OS can perform **critical operations** such as:

- Managing memory
- Handling interrupts
- Controlling input/output devices
- Executing system calls
- Managing processes

Supervisor mode is used to **protect the system** from errors or malicious programs.

If every program had full access to hardware:

- Programs could overwrite system memory
- Devices could be misused
- The system might crash

So the OS separates execution into **two modes**:

1. **User Mode**
2. **Supervisor Mode**

Features of Supervisor Mode

- Full access to **CPU instructions**
- Direct access to **memory and hardware**
- Can execute **privileged instructions**
- Used only by **Operating System kernel**

Example

Suppose a user program wants to **read data from a disk**.

Steps:

1. The **user program runs in User Mode**.
2. It makes a **system call** to request disk access.
3. The CPU **switches to Supervisor Mode**.
4. The **Operating System performs the disk operation**.
5. After completion, the CPU **returns to User Mode**.

This ensures **security and system stability**.

---

5. Simple Diagram

```
+---------------------+
|   User Programs     |
```

```
|     (User Mode)       |
+----------|----------+
           | System Call
           v
+---------------------+
|   Operating System  |
|   (Supervisor Mode) |
+---------------------+
```

Supervisor Mode is the mode in which the Operating System runs and has complete control over the computer system and hardware.

The **Producer–Consumer Problem** is a classic problem in **Operating Systems** used to demonstrate **process synchronization**. One or more **producers** produce data and place it in a **buffer**, while one or more **consumers** take data from the buffer. The main issue is to ensure that both do not access the buffer simultaneously and cause errors.

This problem is commonly solved using **semaphores**.

Classical **Inter-Process Communication (IPC) problems** are used in Operating System to explain **process synchronization** and **resource sharing**. These problems show how multiple processes communicate safely without causing errors like **race conditions, deadlock, or starvation**.

1. Producer–Consumer Problem

The **Producer–Consumer problem** is also called the **Bounded Buffer Problem**.

- A **Producer** process produces data (items).
- A **Consumer** process consumes the data.
- Data is stored in a **shared buffer** of fixed size.
- The producer must **not add items when the buffer is full**.
- The consumer must **not remove items when the buffer is empty**.

Example

Suppose:

- Buffer size = 5
- Producer produces items A, B, C, D
- Consumer consumes items from the buffer.

Flow:

1. Producer inserts item into buffer.
2. Consumer removes item.
3. Synchronization ensures safe access.

Semaphores Used

- **mutex** → ensures mutual exclusion
- **empty** → number of empty buffer slots
- **full** → number of filled buffer slots

Initial values:

```
mutex = 1
empty = N   (buffer size)
full = 0
```

Producer Algorithm

```
do
{
   produce item

   wait(empty)
   wait(mutex)

   add item to buffer

   signal(mutex)
   signal(full)

} while(true)
```

Consumer Algorithm

```
do
{
   wait(full)
   wait(mutex)

   remove item from buffer

   signal(mutex)
   signal(empty)

   consume item

} while(true)
```

Simple Example

Buffer size = 3

Step	Buffer	Action
1	Empty	Producer adds A
2	A	Producer adds B

Step	Buffer	Action
3	A B	Consumer removes A
4	B	Producer adds C

This continues safely without conflict.

2. Readers–Writers Problem

This problem occurs when:

- Multiple processes **read and write a shared database/file**.
- **Readers** only read data.
- **Writers** modify data.

Rules:

1. Many readers can read **simultaneously**.
2. Only **one writer** can write at a time.
3. When a writer writes, **no reader can access the data**.

Example

Shared database: Student Records

Processes:

- Reader → View student details
- Writer → Update student marks

Possible situation:

```
Reader1 → reading
Reader2 → reading
Writer → must wait
```

When readers finish:

```
Writer → writing
```

Variables

```
readcount = 0
mutex = 1
wrt = 1
```

Reader Algorithm

```
wait(mutex)
readcount = readcount + 1

if(readcount == 1)
   wait(wrt)

signal(mutex)

READ DATA

wait(mutex)
readcount = readcount - 1

if(readcount == 0)
   signal(wrt)

signal(mutex)
```

Writer Algorithm

```
wait(wrt)

WRITE DATA

signal(wrt)
```

Example Flow

Step	Process	Action
1	Reader1	Starts reading
2	Reader2	Reads simultaneously
3	Writer	Waits
4	Readers finish	Writer starts writing

---

3. Dining Philosophers Problem

This problem was introduced by Edsger W. Dijkstra.

- Five philosophers sit around a round table.
- Each philosopher alternates between **thinking and eating**.
- There is **one fork between each philosopher**.
- To eat, a philosopher needs **two forks**.

Problem:
If every philosopher picks **one fork**, they may **wait forever → deadlock**.

Example

Philosophers: P1, P2, P3, P4, P5
Forks: F1, F2, F3, F4, F5

Table arrangement:

```
P1  F1  P2  F2  P3  F3  P4  F4  P5  F5
```

Each philosopher needs:

```
Left fork + Right fork
```

Basic Algorithm

```
do
{
   think

   wait(left fork)
   wait(right fork)

   eat

   signal(left fork)
   signal(right fork)

} while(true)
```

Deadlock Situation

All philosophers pick **left fork first**.

```
P1 holds F1 waiting F2
P2 holds F2 waiting F3
P3 holds F3 waiting F4
P4 holds F4 waiting F5
P5 holds F5 waiting F1
```

All wait → **Deadlock**

Deadlock Solution

One solution:

Allow only **4 philosophers to sit at the table at a time**.

Semaphore:

```
room = 4
```

Algorithm:

```
wait(room)

wait(left fork)
wait(right fork)

eat

signal(left fork)
signal(right fork)

signal(room)
```

This prevents deadlock.

Comparison of IPC Problems

Problem	Main Issue	Example
Producer–Consumer	Buffer synchronization	Data sharing
Readers–Writers	Database access	Multiple readers
Dining Philosophers	Resource allocation	Deadlock

Key Concepts Demonstrated

These problems explain:

- **Mutual Exclusion**
- **Deadlock**
- **Starvation**
- **Process Synchronization**
- **Semaphore usage**

Problem	Focus
Producer Consumer	Shared Buffer
Readers Writers	Shared Data
Dining Philosophers	Deadlock

Readers-Writers Problem

The Readers-Writers Problem is a classic synchronization problem where multiple processes are involved in reading and writing data from a shared resource. This problem typically involves two types of processes:

- **Readers**: These processes only read data from the shared resource and do not modify it.
- **Writers**: These processes modify or write data to the shared resource.

The challenge in the Reader-Writer problem is to allow multiple readers to access the shared data simultaneously without causing issues. However, only **one writer** should be allowed to write at a time, and no reader should be allowed to read while a writer is writing. This ensures the integrity and consistency of the data.

Dining Philosophers Problem

The **Dining Philosophers Problem** is a well-known example that shows the difficulties of sharing resources and preventing deadlock when multiple processes are involved. The problem involves a set of philosophers sitting around a dining table. Each philosopher thinks deeply, but when they are hungry, they need to eat. However, to eat, they must pick up two forks from the table, one from the left and one from the right.

Problem Setup:

- There are **five philosophers** sitting around a circular table.
- Each philosopher has a plate of food in front of them and a fork to their left and right.
- A philosopher needs both forks to eat. If they pick up a fork, they hold it until they finish eating.
- After eating, they put down both forks and start thinking again.

The problem arises when multiple philosophers try to pick up forks at the same time, which can lead to a situation where each philosopher holds one fork but cannot get the second fork, leading to a **deadlock**. Additionally, there's a risk of **starvation** if some philosophers are continually denied the opportunity to eat.

Dining Philosophers Problem – Solution (using Semaphores)

Advantages of Process Synchronization

- Ensures data consistency and integrity
- Avoids race conditions

- Prevents inconsistent data due to concurrent access
- Supports efficient and effective use of shared resources

Disadvantages of Process Synchronization

- Adds overhead to the system
- This can lead to performance degradation
- Increases the complexity of the system
- Can cause deadlock if not implemented properly.

Semaphores in Process Synchronization

[redacted]

A semaphore is a synchronization tool that manages how processes share resources. Semaphores are used to control access to critical sections of code, and to prevent deadlocks.

Semaphores refer to the integer variables that are primarily used to solve the critical section problem via combining two of the atomic procedures, wait and signal, for the process synchronization

Semaphores are a tool used in operating systems to help manage how different processes (or programs) share resources, like memory or data, without causing conflicts.
A semaphore is a special kind of synchronization data that can be used only through specific synchronization primitives.

Semaphores are used to implement critical sections,
which are regions of code that must be executed by only one process at a time. By using semaphores, processes can coordinate access to shared resources, such as shared memory or I/O devices.

Semaphores

A semaphore is a synchronization tool used in concurrent programming to manage access to shared resources.
It is a lock-based mechanism designed to achieve process synchronization, built on top of basic locking techniques.
Semaphores use a counter to control access, allowing synchronization for multiple instances of a resource.
Processes can attempt to access one instance, and if it is not available, they can try other instances.

Unlike basic locks, which allow only one process to access one instance of a resource.
Semaphores can handle more complex synchronization scenarios, involving multiple processes or threads.
It help prevent problems like **race conditions** by controlling when and how processes access shared data.

The process of using Semaphores provides two operations:

- **wait (P):** The wait operation decrements the value of the semaphore
- **signal (V):** The signal operation increments the value of the semaphore.

When the value of the semaphore is zero, any process that performs a wait operation will be blocked until another process performs a signal operation.
When a process performs a wait operation on a semaphore, the operation checks whether the value of the semaphore is >0.
If so, it decrements the value of the semaphore and lets the process continue its execution; otherwise, it blocks the process on the semaphore.
A signal operation on a semaphore activates a process blocked on the semaphore if any, or increments the value of the semaphore by 1. Due to these semantics, semaphores are also called counting semaphores.

The initial value of a semaphore determines how many processes can get past the wait operation.
Semaphores are required for process synchronization to make sure that multiple processes can safely share resources without interfering with each other. They help control when a process can access a shared resource, preventing issues like **race** conditions.

Types of Semaphores

Semaphores are of two Types:

- **Binary Semaphore:** This is also known as a **mutex lock**, as they are locks that provide mutual exclusion. It can have only two values – 0 and 1. Its value is initialized to 1. **It is used to implement the solution of critical section problems with multiple processes and a single resource**.
- **Counting Semaphore:** Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.

The semaphore is initialized to the number of resources available. Its value can range over an unrestricted domain.

Working of Semaphore

A semaphore is a simple yet powerful synchronization tool used to manage access to shared resources in a system with multiple processes. It works by maintaining a counter that controls access to a specific resource, ensuring that no more than the allowed number of processes access the resource at the same time.

There are two primary operations that a semaphore can perform:

1. **Wait (P operation)**:
   - This operation checks the semaphore's value.
   - If the value is greater than 0, the process is allowed to continue, and the semaphore's value is decremented by 1.
   - If the value is 0, the process is blocked (waits) until the semaphore value becomes greater than 0.
2. **Signal (V operation)**:
   - After a process is done using the shared resource, it performs the signal operation.
   - This increments the semaphore's value by 1, potentially unblocking other waiting processes and allowing them to access the resource.

```
P(Semaphore s){
    while (S == 0);    /* wait until s = 0 */
  s = s - 1;
}

V(Semaphore s){
        s = s + 1;
}
```

Note that there is Semicolon after while. The code gets stuck Here while s is 0.

A critical section is surrounded by both operations to implement process synchronization. The below image demonstrates the basic mechanism of how semaphores are used to control access to a critical section in a

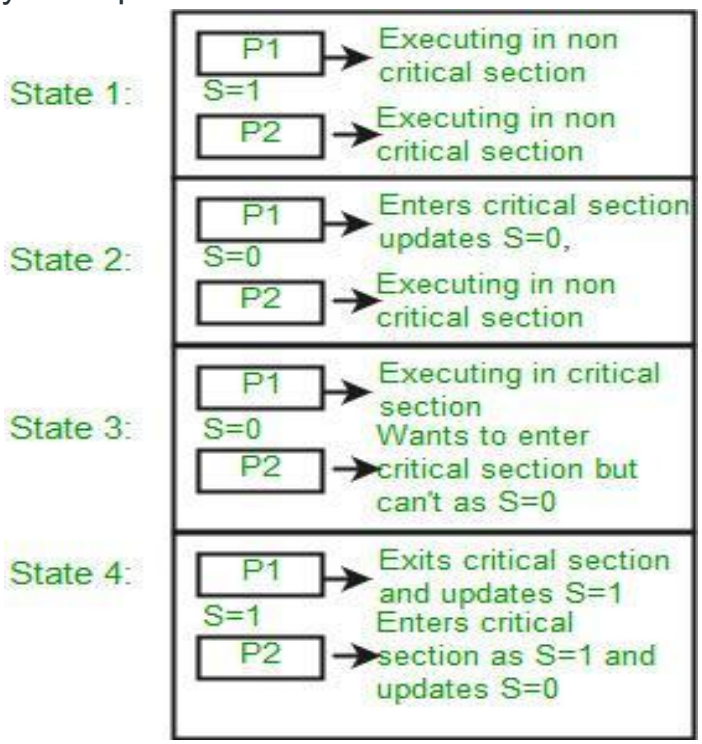
multi-process environment, ensuring that only one process can access the shared resource at a time

```
Process P

// Some code
P(s);
  // critical section
V(s);
  // remainder section
```

Now, let us see how it implements mutual exclusion. Let there be two processes P1 and P2 and a semaphore s is initialized as 1. Now if suppose P1 enters in its critical section then the value of semaphore s becomes 0. Now if P2 wants to enter its critical section then it will wait until s > 0, this can only happen when P1 finishes its critical section and calls V operation on semaphore s.

This way mutual exclusion is achieved. Look at the below image for details which is a Binary semaphore.



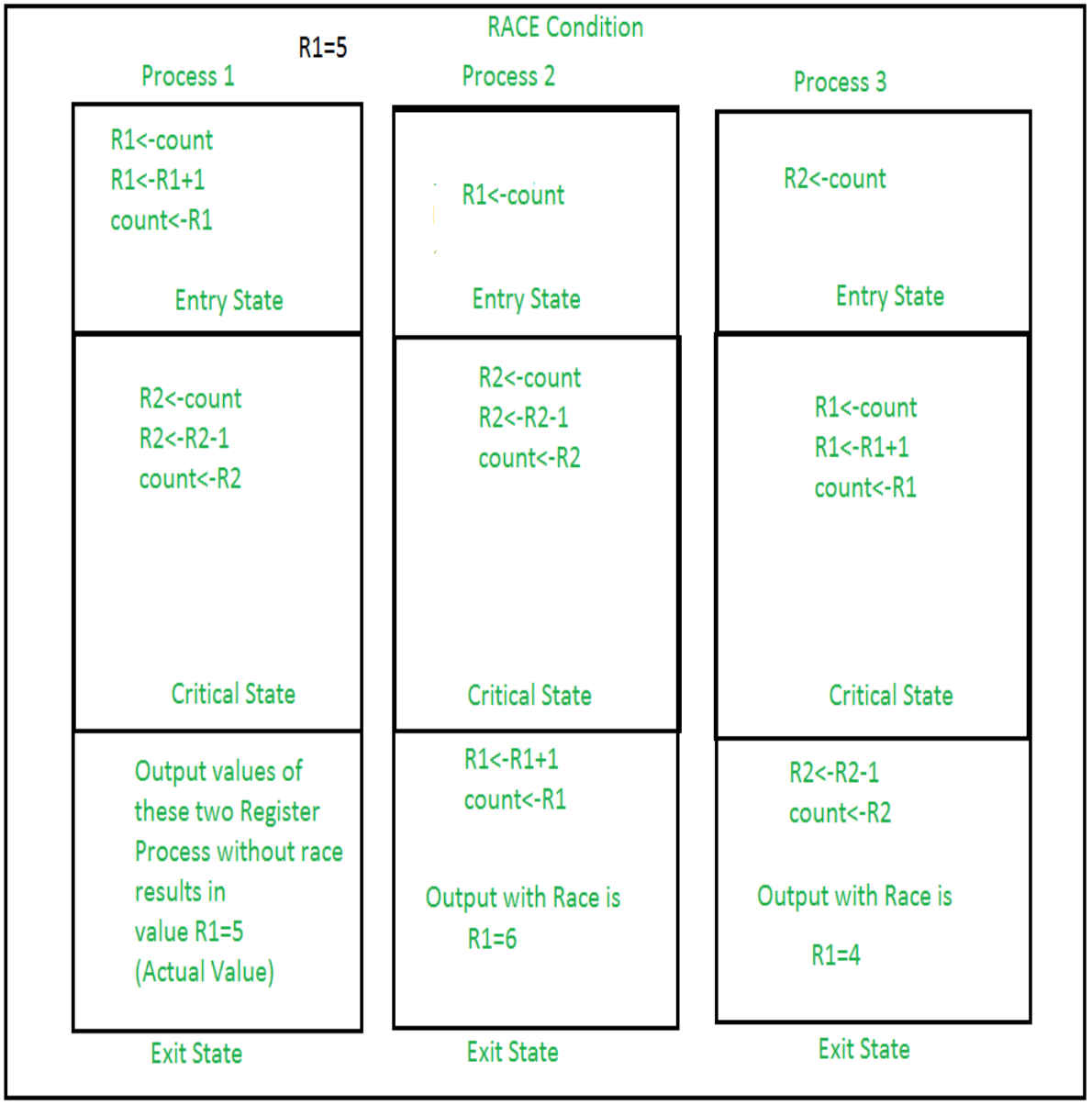
Process Synchronization

[redacted]

Process Synchronization is a technique which is used to coordinate the process that use shared Data. There are two types of Processes in an Operating Systems:-

1. **Independent Process –** The process that does not affect or is affected by the other process while its execution then the process is called Independent Process. Example The process that does not share any shared variable, database, files, etc.
2. **Cooperating Process –** The process that affect or is affected by the other process while execution, is called a Cooperating Process. Example The process that share file, variable, database, etc are the Cooperating Process.

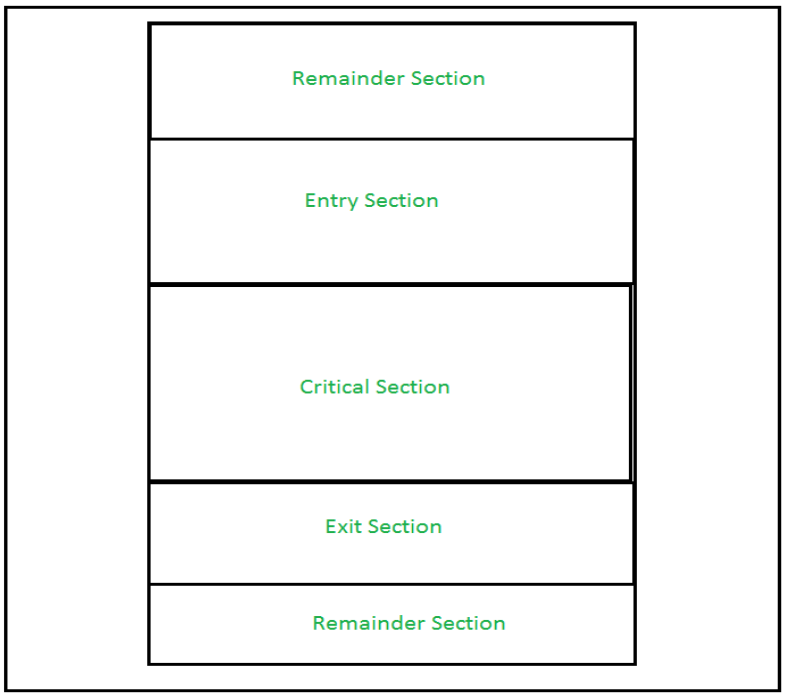
We have already introduced Different Terms in Process Synchronization. Process Synchronization is mainly used for Cooperating Process that shares the resources.Let us consider an example of //racing condition image



It is the condition where several processes tries to access the resources and modify the shared data concurrently and outcome of the process depends on the particular order of execution that leads to data inconsistency, this condition is called

Race Condition : This condition can be avoided using the technique called Synchronization or Process Synchronization, in which we allow only one process to enter and manipulates the shared data in Critical Section.

//diagram of the view of CS



This setup can be defined in various regions like:

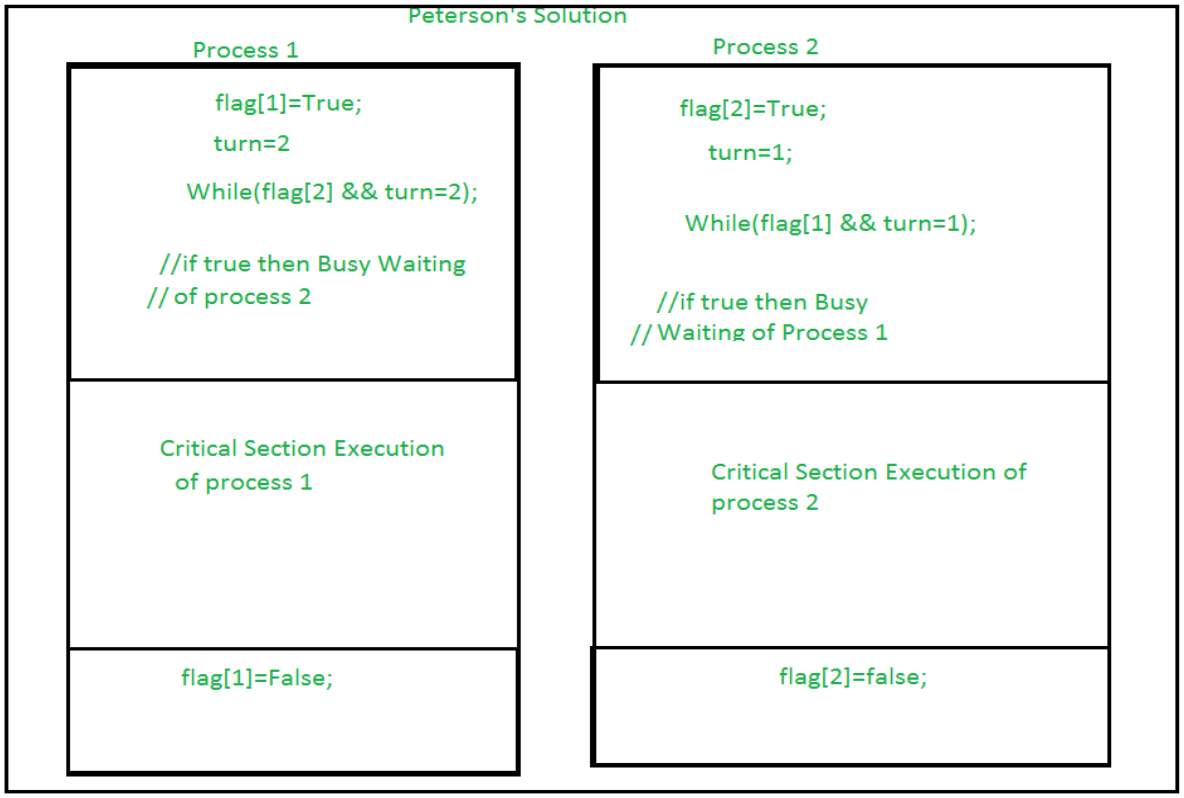
- **Entry Section –** It is part of the process which decide the entry of a particular process in the Critical Section, out of many other processes.
- **Critical Section –** It is the part in which only one process is allowed to enter and modify the shared variable.This part of the process ensures that only no other process can access the resource of shared data.
- **Exit Section –** This process allows the other process that are waiting in the Entry Section, to enter into the Critical Sections. It checks that a process that after a process has finished execution in Critical Section can be removed through this Exit Section.
- **Remainder Section –** The other parts of the Code other than Entry Section, Critical Section and Exit Section are known as Remainder Section.

Critical Section problems must satisfy these three requirements:

1. **Mutual Exclusion –** It states that no other process is allowed to execute in the critical section if a process is executing in critical section.
2. **Progress –** When no process is in the critical section, then any process from outside that request for execution can enter in the critical section without any delay. Only those process can enter that have requested and have finite time to enter the process.
3. **Bounded Waiting –** An upper bound must exist on the number of times a process enters so that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

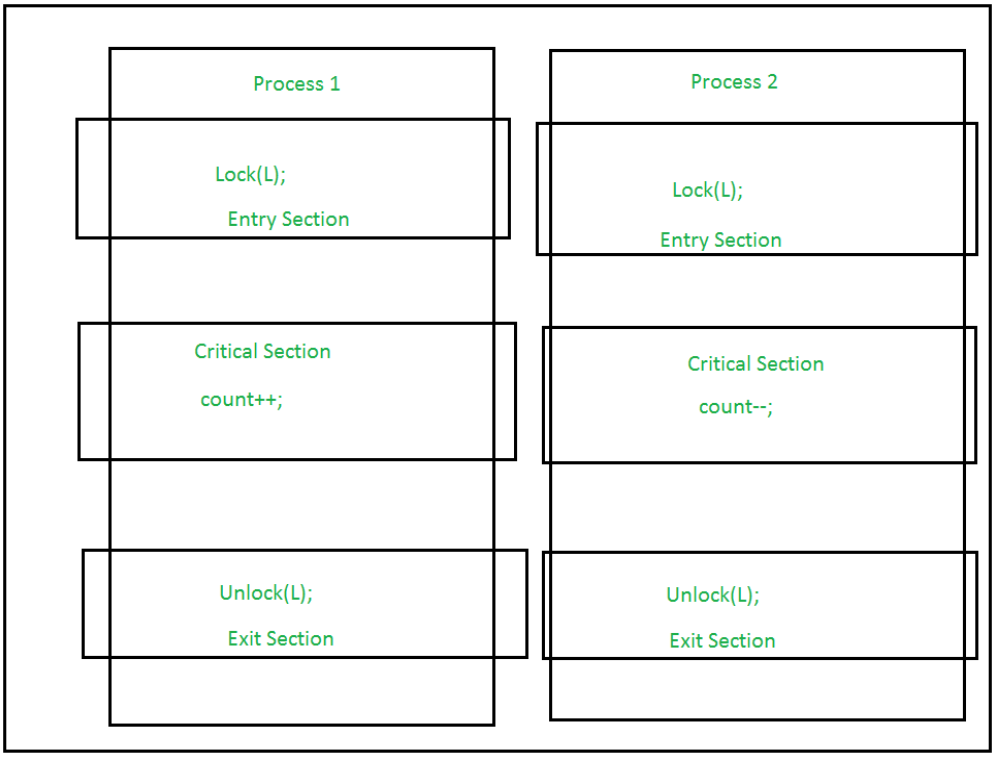
Process Synchronization are handled by two approaches:

1. **Software Approach –** In Software Approach, Some specific Algorithm approach is used to maintain synchronization of the data. Like in Approach One or Approach Two, for a number of two process, a temporary variable like (turn) or boolean variable (flag) value is used to store the data. When the condition is True then the process in waiting State, known as Busy Waiting State. This does not satisfy all the Critical Section requirements. Another Software approach known as Peterson's Solution is best for Synchronization. It uses two variables in the Entry Section so as to maintain consistency, like Flag (boolean variable) and Turn variable(storing the process states). It satisfy all the three Critical Section requirements. //Image of Peterson's Algorithm



2. **Hardware Approach –** The Hardware Approach of synchronization can be done through Lock & Unlock technique.Locking part is done in the Entry Section, so that only one process is allowed to enter into the Critical Section, after it complete its execution, the process is moved to the Exit Section, where Unlock Operation is done so that another process in the Lock Section can repeat this process of Execution.This process is designed in such a way that all the three conditions of the

Critical Sections are satisfied. //Image of Lock



Synchronization Examples

[redacted]

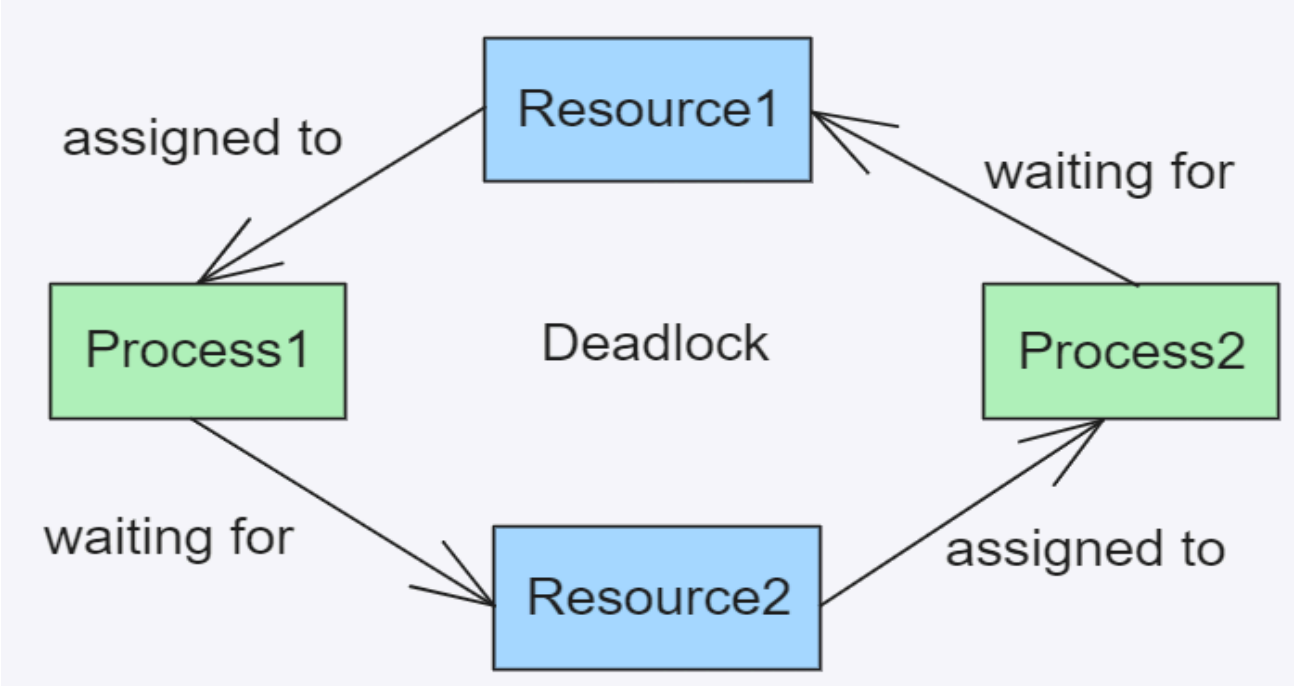
The Synchronization is an important concept in operating systems that ensures the smooth and coordinated execution of processes and threads. It is the task of coordinating the execution of processes in such a way that no two processes can access the same shared data and resource. It is a critical part of operating system design which ensures that processes or threads can safely share resources without interfering with each other. Synchronization is important because it helps in avoiding Data inconsistency between multiple processes or threads, Deadlocks and Race conditions. It is generally used in multi-processing systems, where process concurrently attempt to access the same shared resource of piece of data.

Primary Terminologies

Deadlock

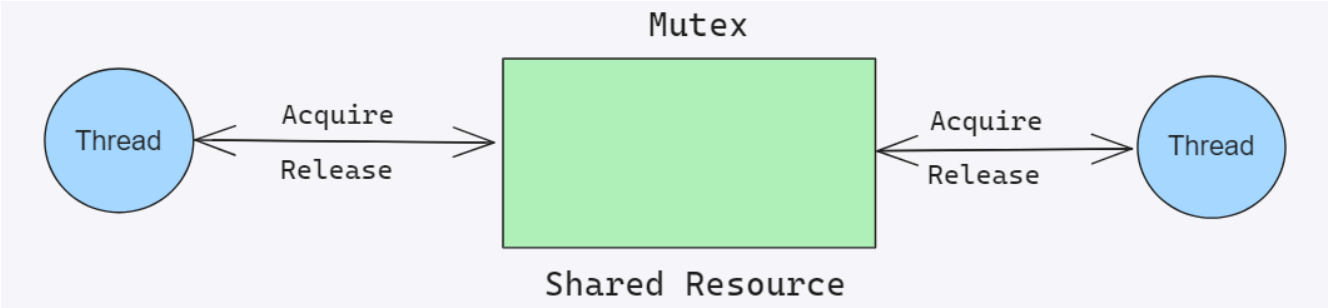
Deadlock is a bug present in process or thread synchronization method. It is a condition where two or more processes are waiting for some resource availability which will never be available as it is required by another process

which is busy with some other processes. These are said to be in a deadlock situation.



Mutex

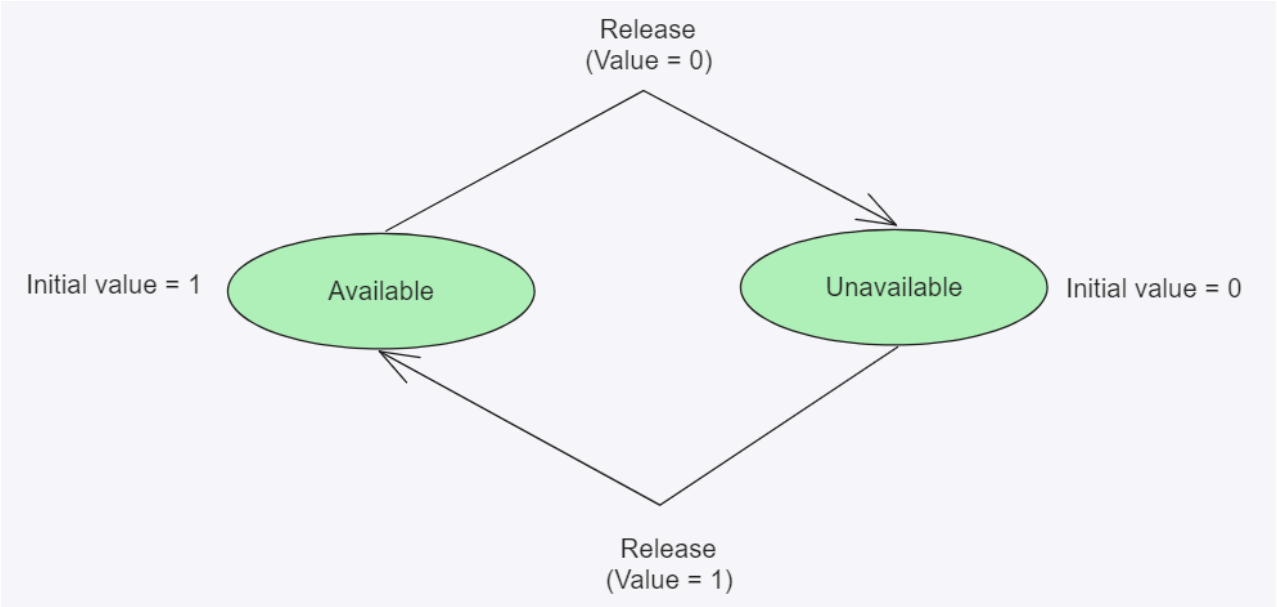
Mutex means "mutual exclusion". It is synchronization primitive that is used to prevent multiple threads to simultaneously access shared resources. Only one thread or process can enter the critical section of code at a time. So, it ensures that resources are not accessed by multiple processes or threads concurrently.



Semaphores

It is a synchronization method. It is an integer value that is equal to number of resources. It is used to control access to a shared resource by multiple processes or threads. It maintains a counter and when counter reaches zero then further attempts to acquire the semaphore are blocked until its value become greater than zero. These are of two types:

- Binary semaphore
- Counting semaphore



Race Condition

Race condition occurs when two or more threads or processes can access the shared data and try to change it at same time. Because in thread scheduling algorithms, we can swap between threads at any time. We do not know the order in which threads attempt to access the shared data and change the data. So, both threads are racing to access and change the data cause the race condition.

Critical Section

Refers to segment of the code where processes/threads access shared resources such as common variable, file and perform read/write operation on them. Mutex and semaphore methods are used to ensure that only one thread/process can access critical section at a time.

Examples

Producer-Consumer Problem

It is also known as bounded buffer problem. Here, we have two threads one is producer thread and other is consumer thread. Critical section (buffer) is a shared resource. We do not want that inconsistency will be produced so we want synchronization between producer and consumer thread.

- Producer must not insert the data when the buffer is full.
- Consumer must not pick/remove data when the buffer is empty.

Solution (with the help of semaphores) => we will use both binary and counting semaphores here. So, we will take as:

- m, mutex => used to acquire lock on buffer.
- empty => a counting semaphore with initial value is n. Used to track empty slots.
- full => tracks filled slots and initial value is zero.

```
Producer  solution

do{
      wait(empty); // wait until empty > 0 then empty->value

      wait(mutex);
      // critical section => add data to buffer
      signal(mutex);
      signal(full); // increment full->value
}while(true)
Consumers solution

do{
      wait(full); // wait until full > 0 then fill --

      wait(mutex);
      //remove data from buffer
      Signal(mutex);
      Signal(empty);
      // increment empty
} while(true)
```

Reader-Writer Problem

Here, we want synchronization that at one time only one writer can write and multiple readers can read. Here, we have two threads one is Read and other is Write/Update.

- If there are more than one readers reading then there is no issue.
- If there are more than one writers or one writer and some other thread(r/w) occur parallelly then Race condition will arise there and data will become inconsistent.

Solution (using semaphores)

- mutex (binary semaphore) => to ensure mutual exclusion, when read count (rc) is updated. No two threads modify read count at same time.
- wrt (write) => Binary semaphore common for both reader and writer.
- read count (rc) => integer = {0} initially, tracks how many readers are reading in critical section.

```
Writer solution

do {
      wait(wrt);
      // do write operation
      signal(wrt);
} while(true);
```

Reader Solution

do {

```
wait(mutex); // to mutex read count unable
rc++;
if(rc == 1)
wait(wrt) // ensure no writer thread can enter if there is even one reader
signal(mutex)
// critical section => reader is reading
wait(mutex);
rc--; // a reader leaves
if(rc == 0) // no reader is left in critical section
signal(wrt) // writer can enter into critical section now
signal(mutex) // reader leaves
} while(true)
```

Classical Problems of Synchronization with Semaphore Solution

[redacted]

With the evolution of multi-processors, tasks suffer due to overheads and traffic. To manage the resources efficiently, synchronization is needed. Synchronization ensures perfect execution of processes and controlled access to shared resources. Synchronization in operating systems is often explained with the help of real-life examples.

In this article, we will see a number of classical problems of synchronization as examples of a large class of concurrency-control problems. In our solutions to the problems, we use semaphores for synchronization, since that is the traditional way to present such solutions. However, actual implementations of these solutions could use mutex locks instead of binary semaphores.

Synchronization Problems with Semaphore Solution

These problems are used for testing nearly every newly proposed synchronization scheme. The following problems of synchronization are considered as classical problems:

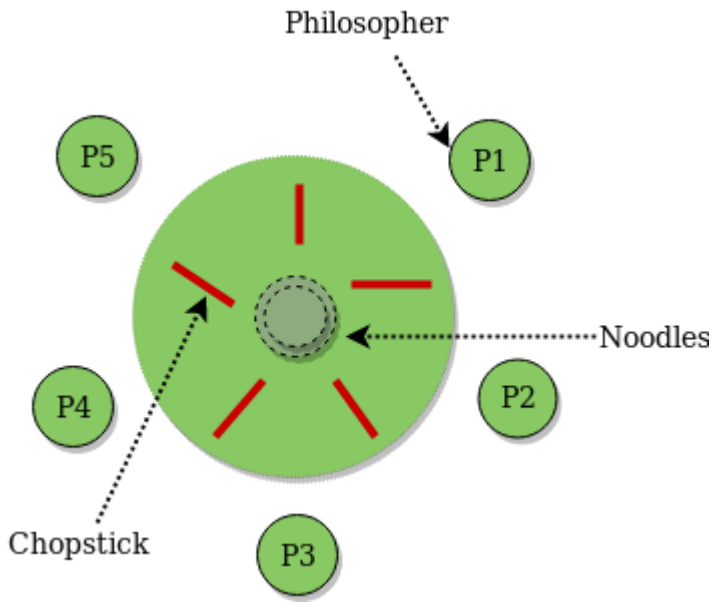
```
1. Bounded-buffer (or Producer-Consumer) Problem,
2. Dining-Philosophers Problem,
3. Readers and Writers Problem,
4. Sleeping Barber Problem
```

Bounded-Buffer (or Producer-Consumer) Problem

The Bounded Buffer problem is also called the producer-consumer problem. This problem is generalized in terms of the Producer-Consumer problem. The solution to this problem is, to create two counting semaphores “full” and “empty” to keep track of the current number of full and empty buffers respectively. Producers produce a product and consumers consume the product, but both use of one of the containers each time.

Dining-Philosophers Problem

The Dining Philosopher Problem states that K philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pickup the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both. This problem involves the allocation of limited resources to a group of processes in a deadlock-free and starvation-free manner.

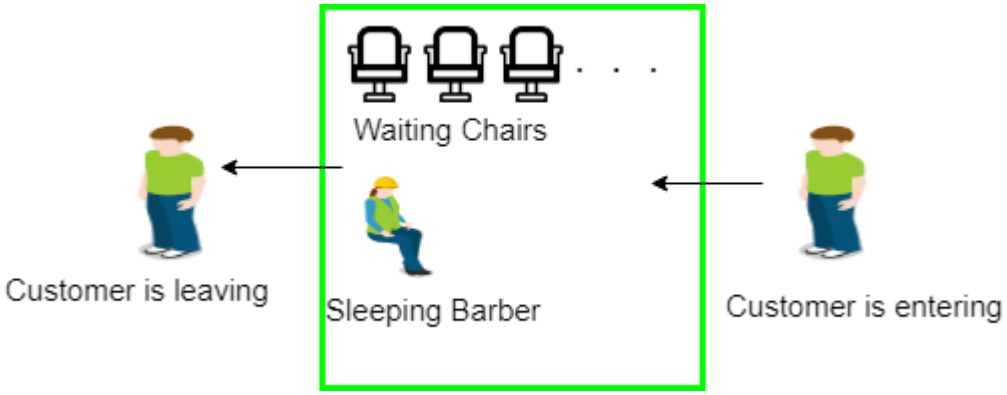


Readers-Writers Problem

Suppose that a database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database. We distinguish between these two types of processes by referring to the former as readers and to the latter as writers. Precisely in OS we call this situation as the readers-writers problem. Problem parameters:

- One set of data is shared among a number of processes.
- Once a writer is ready, it performs its write. Only one writer may write at a time.
- If a process is writing, no other process can read it.
- If at least one reader is reading, no other process can write.

- Readers may not write and only read.

Sleeping Barber Problem

- Barber shop with one barber, one barber chair and N chairs to wait in. When no customers the barber goes to sleep in barber chair and must be woken when a customer comes in. When barber is cutting hair new customers take empty seats to wait, or leave if no vacancy. This is basically the [Sleeping Barber Problem.]